DELIVERABLE 03 / IMPLEMENTED TECHNICAL DESIGN

Technical Approach Report

Cynbit AutoBlueprint

A practical explanation of the working prototype: how an inquiry becomes a sourced technical draft, how quality is checked, and where a human architect remains accountable.

PREPARED BY
Sudeep Sagar

ASSESSMENT
AI/ML Engineer Intern

SOURCE CODE
github.com/sudeepsagar123/cynbit-autoblueprint

LIVE DEMO
cynbit-autoblueprint-jfh5jn799yjuhrq3zips2k.streamlit.app

Executive summary

Cynbit AutoBlueprint is a staff-facing prototype that turns a structured project inquiry and free-text brief into an initial, source-linked technical blueprint for human review. It combines a Researcher agent, an Architect agent, a Critic agent, a retrieval pipeline over public Cynbit material, a bounded revision loop and an explicit approval step.

The design decision

Automate preparation of the first draft, not the architect's decision. The system can organise requirements, find relevant public references and propose an architecture. A human must review assumptions, estimates, risks and any content intended for a prospect.

INPUT	PROCESS	OUTPUT
Customer / company details, project goals, constraints and description.	Validate → retrieve → draft → critique → revise if needed.	18-section draft, evidence links, approval record and PDF export.

Implementation boundary: the submitted prototype uses public Cynbit information and synthetic demonstration inquiries. It is not connected to Cynbit's internal CRM, private case-study repository or production delivery systems.

01 / PROBLEM FRAMING

Business context, users and scope

The technical design starts from a publicly described customer journey, while keeping unknown internal operations explicitly open for validation.

Why this workflow was selected

Cynbit's Contact experience describes "Project Intelligence - The Blueprint": a prospect explains what they want to build, and Cynbit analyses the requirements and prepares a tailored strategic overview before connecting. Cynbit's Services page also describes autonomous agents, LLM automation and document handling as service capabilities [1][2].

AutoBlueprint is proposed as an internal drafting assistant for that step. The public pages establish the customer-facing outcome, but do not disclose inquiry volumes, internal architect effort, private data structures, current tools or process bottlenecks. Those must be confirmed with the team before an operational business case is made.

Intended users and jobs to be done

User	What they need	How the prototype supports it
Business / discovery team	Turn an inquiry into an organised brief; know what is missing.	Structured fields, requirement extraction and a list of clarification points.
Solution architect	Start from a traceable draft rather than a blank page.	Public reference retrieval, architecture outline, stack rationale, API/data concepts and explicit assumptions.
Reviewer / approver	Check quality and retain decision authority.	Critic findings, revision history, edit controls and explicit approval status.

Prototype scope

Included in this implementation	Not included / not claimed
Manual inquiry entry in Streamlit; schema validation; prompt-injection pattern sanitisation.	Automatic ingestion from Cynbit's live contact form, CRM, email or sales system.
Retrieval from a curated corpus of 16 public Cynbit records with source URLs.	Private internal case studies, customer files, code repositories or confidential client records.
Researcher, Architect and Critic agents; structured output; up to two revision attempts.	Autonomous customer communication, commercial commitments or unattended architecture sign-off.
Human review states, SQLite persistence and PDF generation.	Production identity management, role-based access control, multi-tenant isolation or managed database service.

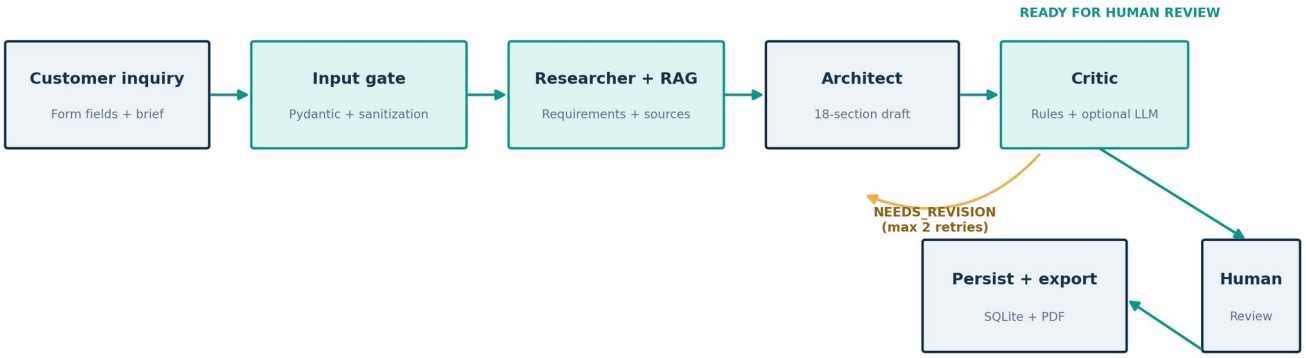
Important terminology

"Blueprint generation" means a first-pass technical proposal. It does not mean code generation, a production architecture certification or an automatically approved customer proposal. All estimates are preliminary engineering assumptions.

02 / SYSTEM DESIGN

End-to-end architecture

The workflow is intentionally sequential and observable. The Critic can send a draft back to the Architect, but the loop is bounded.



Only a human reviewer can set APPROVED, REJECTED or REVISION_REQUESTED.

What happens at each stage

Stage	Input → work performed → output
1. Input gate	Receives inquiry fields. Pydantic validates required fields and types; a sanitisation step detects and neutralises selected prompt-injection patterns. Invalid input does not proceed.
2. Researcher + RAG	Builds a query from industry, title, description and objectives. Extracts requirements and missing information; retrieves up to three case-study references and up to two product / service references.
3. Architect	Uses the inquiry and retrieved context to draft the 18 blueprint sections, technology rationale, Mermaid diagram, proposed schemas / APIs, phases, assumptions and risks.
4. Critic	Checks required sections, requirement mapping, security content, estimate disclaimers and reference URLs. Optional Gemini critique adds another review when configured and deterministic checks pass.
5. Revision route	If the result needs correction and retry budget remains, the graph returns to Architect. The configured maximum is two retries; execution is bounded.
6. Human gate	A blueprint is staged as PENDING_APPROVAL. A reviewer can approve, reject, edit or request revision. The decision and notes are persisted.

The orchestration uses LangGraph StateGraph when the package is available. An API-compatible fallback graph is included for minimal offline execution. The graph has explicit nodes for validation, research, architecture, critique and human review.

03 / AGENT RESPONSIBILITIES

What each agent actually does

Specialisation keeps each prompt focused and makes the workflow easier to inspect and test.

Component	Responsibility	Checks / output
Input validator & security gate	Validate inquiry fields and clean selected hostile instruction patterns from user-provided text.	ProjectInquiryInput schema; validation errors; security flags.
Researcher agent	Classify domain; extract business objectives, functional and non-functional requirements, constraints and missing information; call RAG.	ResearchFindings; retrieved case studies and reusable product/service references; URLs and backend metadata.
Architect agent	Convert findings into a structured architecture draft.	18 named sections, requirement mapping, solution diagram, stack with rationale, database / API concepts, phases, risks and assumptions.
Critic agent	Apply deterministic quality checks; optionally use Gemini for critique after hard checks pass.	Requirement coverage, section completeness, security checks, estimate qualification, URL checks, risks and recommendation.
Human approval manager	Keep the proposed blueprint pending until a reviewer makes a decision.	APPROVED, REJECTED or REVISION_REQUESTED; reviewer name, notes and timestamp.

The 18-section blueprint

1 Project overview	10 API and integration requirements
2 Customer business problem	11 Relevant Cynbit case studies
3 Project objectives	12 Implementation phases
4 Extracted functional requirements	13 Preliminary timeline and effort estimates
5 Non-functional requirements	14 Infrastructure requirements
6 Proposed solution	15 Security considerations
7 Recommended system architecture	16 Technical risks and mitigation
8 Technology stack	17 Assumptions and missing information
9 Database design	18 Recommended next steps

How the Critic decides

The Critic computes requirement coverage by comparing extracted functional requirements with the Architect's mapped requirement items. It checks that all required blueprint sections are present, that the security section contains enough concrete controls, that effort estimates are explicitly labelled preliminary assumptions, and that cited Cynbit URLs use the verified public domain. If hard checks identify corrections, the Architect receives a revision request while retry budget remains.

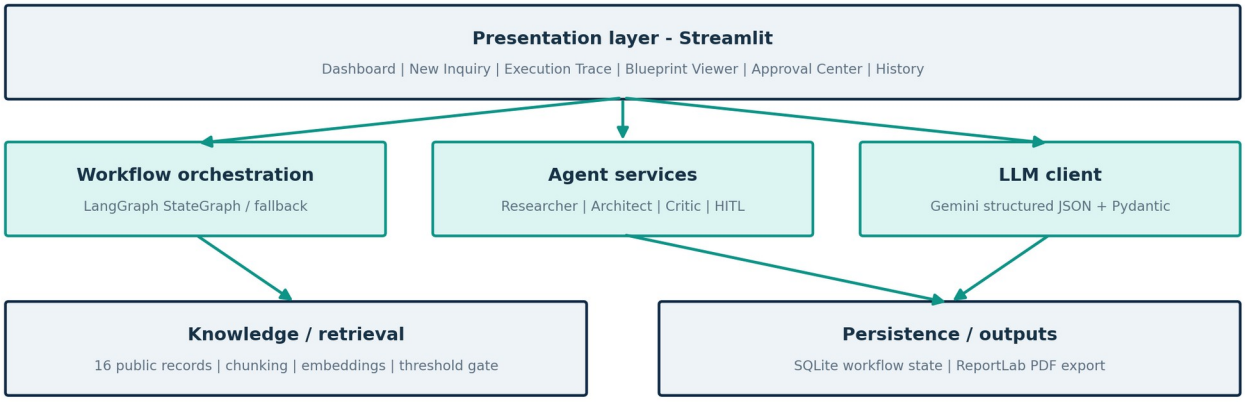
Design intent

The Critic is a quality gate, not proof of correctness. A passing draft still requires a qualified human architect to validate scope, feasibility, security, cost and delivery assumptions.

04 / RETRIEVAL AND GROUNDING

How the knowledge base reduces unsupported claims

RAG provides reference material; it does not turn public case-study descriptions into access to Cynbit's private project data.



External dependency: Google Gemini API when a valid API key is configured. Offline fallback remains available.

Knowledge base composition

The bundled corpus contains 16 curated public records from Cynbit's website: public case-study listings, product pages, the Services technology-stack description, and Cynbit's article on AI agents and human-in-the-loop governance [2][3][4]. The current ingestion module deliberately limits case-study evidence to publicly stated titles, categories and featured text; it does not invent confidential per-project technology stacks or metrics.

Source group	Use in the prototype	Evidence boundary
Case-study listings	Find public examples with similar domain or problem language.	Only what the public page exposes is treated as evidence.
Product pages: TenantEase, Revio, Cynbit Studio	Identify public product capabilities that may be reusable or relevant.	Product marketing statements are not independently validated product internals.
Services page	Ground broad technology and service recommendations.	Does not prove that a particular stack was used in a specific client project.
AI agents engineering article	Inform agent / HITL design principles.	A published article is design context, not proof of this prototype's production security.

Retrieval pipeline

- Ingestion and cleaning: curated records are normalised and split into metadata-bearing chunks.
- Embedding selection: Gemini text-embedding-004 is attempted when a key is configured; otherwise Sentence Transformers all-MiniLM-L6-v2 is attempted when installed; the deterministic hybrid fallback is available when neither is usable.
- Offline fallback: token expansion uses domain synonyms and sublinear TF-IDF, combined with a compact SVD representation. Vectors are L2-normalised for dot-product cosine ranking.
- Ranking and gating: query and chunk vectors are ranked; results are deduplicated by document ID and filtered by a minimum similarity threshold. The retrieval method enforces a floor of 0.25.
- Grounded context: accepted references are passed into the Researcher and Architect prompts with document ID, title, score and public URL. When no source clears the threshold, the system says that no verified reference was found.

Why abstention matters

A low-confidence match should not be presented as a verified Cynbit case study. When retrieval returns no accepted source, the system must label recommendations as proposed architecture, not historical Cynbit evidence.

05 / MODEL INTEGRATION AND GOVERNANCE

Structured LLM calls, fallbacks and safety boundaries

The model provides language reasoning; application code supplies schemas, deterministic checks, state transitions and approval controls.

Gemini structured-output path

services/llm_client.py supports the Google GenAI SDK and a direct Gemini v1beta REST fallback. The configured text model defaults to gemini-3.8-flash; the embedding model defaults to text-embedding-004. Agent responses are parsed as JSON and validated against Pydantic schemas for ResearcherLLMOutput, ArchitectLLMOutput and CriticLLMOutput. Invalid output or API errors do not silently become valid results; the client reports the execution mode and falls back to the deterministic offline agent path.

Situation	Application behaviour
Valid API key and successful request	Calls Gemini for structured text output; validates returned JSON against the relevant Pydantic model.
No API key	Uses OFFLINE_DEMO_MODE deterministic requirement extraction / blueprint synthesis.
Model/API error or invalid response	Reports an offline fallback mode and uses deterministic logic where the caller supports fallback.
Embeddings unavailable	RAG routes to its offline hybrid TF-IDF + synonym + SVD engine.

Security and governance controls implemented

- Input schema validation: required fields and minimum lengths are checked before agent execution.
- Prompt-injection handling: selected patterns in the inquiry description are detected and sanitised; this is a defensive layer, not a complete prompt-injection solution.
- Untrusted evidence boundary: retrieved content is provided as context, and source URLs are retained for reviewer inspection.
- Bounded execution: the Critic-to-Architect loop is limited by max_retries (default 2) and the fallback graph has a maximum step limit.
- Human decision gate: generated drafts are staged as PENDING_APPROVAL; only the reviewer action sets an approval status.
- Secret handling: Streamlit Cloud secrets are loaded into environment variables at app startup; .env and secrets files are excluded from source control.

Controls not yet implemented for production

The prototype does not provide authenticated user identities, role-based access control, tenant isolation, a managed secrets lifecycle, immutable audit storage, formal data-retention enforcement or a production security review. Prompt sanitisation alone cannot guarantee safety. Do not submit confidential customer data or treat the public demo as a production tenant environment.

06 / DATA AND USER EXPERIENCE

What is stored, displayed and exported

The app is designed to make work visible to a reviewer rather than hide the reasoning trail inside a single generated answer.

Streamlit application pages

Page	What the user can do
1. Overview Dashboard	See inquiry counts, approval statuses, average processing time and critic scores derived from the local SQLite records.
2. New Project Inquiry	Enter customer, company, industry, project title, description, objectives, budget, expected timeline and additional requirements; load synthetic examples.
3. Multi-Agent Execution	Inspect the Researcher, RAG, Architect, Critic and approval-gate execution trace and review generated findings.
4. Blueprint Viewer	Read the structured 18-section blueprint, inspect the Mermaid architecture diagram and export a PDF.
5. Approval Center	Select a blueprint, edit/review it and record approval, rejection or revision request with reviewer details and notes.
6. Blueprint History	Search stored runs and filter by approval status; inspect saved workflow state and history.

Persistence model

database/db.py uses SQLite and a blueprints table keyed by inquiry_id. It stores customer and project metadata, domain classification, approval status, critic score, requirement coverage, revision count, processing time, reviewer details, created/updated timestamps and the full serialised workflow state (state_json). The app can list, retrieve, save and delete selected blueprint records.

Deployment storage limitation

On Streamlit Community Cloud, this local SQLite file resides in the app container. It is suitable for a demonstration but must not be assumed to be durable across rebuilds or redeployments. A production system needs a managed database, backups, access control and a defined retention policy.

PDF output

The blueprint formatter assembles the generated sections into readable Markdown. The ReportLab exporter turns the reviewed blueprint into a multi-page PDF. The PDF is an export of the draft/reviewed record; export does not itself constitute human approval or delivery to a customer.

07 / VERIFICATION

What has been tested and what the numbers mean

I re-ran the bundled repository tests and RAG benchmark on 31 October 2026. The results below are reproducible prototype checks, not production performance guarantees.

22 / 22	83.33%	91.67%
automated tests passed	mean Precision@3	mean ranked Recall@3

Automated test coverage

Suite	Coverage	Result
RAG pipeline	Ingestion, cleaning, chunking, embedding normalisation, source attribution, unmatched query abstention and benchmark logic.	6 tests passed
Agents	Requirement extraction, prompt-injection handling and 18-section blueprint generation.	3 tests passed
Workflow	End-to-end state progression, graph wiring and bounded Critic-to-Architect revision behaviour.	3 tests passed
HITL / persistence	Approval lifecycle, SQLite persistence and PDF export.	1 test passed
LLM / LangGraph contract	Mocked Gemini structured output, validation, API-failure fallback, embedding and graph contract.	6 tests passed
Database deletion	Delete-one, delete-selected and empty/invalid selection behaviour.	3 tests passed
Total	Re-run of the extracted package on 31 October 2026 (16.85 seconds).	22 / 22 passed

RAG benchmark: read the definitions carefully

The benchmark contains six hand-authored queries and relevance labels over the bundled public Cynbit corpus. In the audit environment the embedding backend was OFFLINE_HYBRID_TFIDF_SVD. Mean Precision@3 was 83.33% among results accepted by the production confidence threshold. Mean ranked Recall@3 was 91.67% across the top-three unique-document ranking before thresholding. Source URL attribution accuracy was 100% among confidence-accepted references.

These metrics are not one standard precision/recall pair

Precision is measured after confidence gating; recall is measured on the candidate top-k ranking before gating. They describe different stages and must be presented with these definitions. The dataset is small and manually labelled, so it is a smoke-test benchmark rather than a statistical production evaluation.

Limitations of current verification

- The Gemini structured-output integration test uses a mocked transport. It verifies schema handling and agent integration, but does not prove a live Gemini end-to-end workflow.
- The revision-loop recovery test uses a synthetic omission and a controlled second pass; it is not a measured customer outcome.
- The six-query RAG benchmark is small. The FinTech query still has a ranking-quality limitation: TenantEase can rank above GlobalPay even though GlobalPay appears in the top three.
- No load, penetration, multi-user concurrency, production availability or long-duration persistence test is claimed.

08 / DEPLOYMENT AND NEXT STEPS

How to run the prototype and what production would require

The public Streamlit instance is a reviewable demonstration deployment, not a production installation inside Cynbit's environment.

Current demonstration deployment

Source code is hosted in the public GitHub repository on the main branch. The Streamlit Community Cloud app runs app.py and reads GEMINI_API_KEY, GEMINI_MODEL and GEMINI_EMBEDDING_MODEL from the app's Secrets configuration. If a valid key is not configured or a model call fails, the app should display its offline demonstration execution mode.

Live demo: https://cynbit-autoblueprint-jfh5jn799yjuhrq3zips2k.streamlit.app/
Source: https://github.com/sudeepsagar123/cynbit-autoblueprint

Local setup

Clone the repository, create a Python virtual environment, run pip install -r requirements.txt, then launch with streamlit run app.py. Run python -m pytest -q from the repository root to execute the automated suite. Full commands and configuration notes are in README.md.

Production-readiness work before internal adoption

Priority	Work required	Why it matters
P0 - Data and access	Use approved private data only; establish authentication, reviewer roles, tenant isolation, retention and deletion policies.	The current demo does not provide production identity or data governance.
P0 - Storage	Move workflow records to managed PostgreSQL or an approved durable database; add backups and migrations.	Local SQLite in a Streamlit container is not a durable multi-user system of record.
P1 - Quality	Build an architect-labelled evaluation set; measure evidence support, requirement coverage, edit rate and failure types.	A six-query benchmark cannot establish production quality.
P1 - Reliability	Add structured logs, error monitoring, API timeouts/retries, rate limits, cost tracking and model/version traceability.	Operational teams need to diagnose failures and control cost.
P1 - Integration	Only after process mapping, design a controlled connector to approved inquiry/CRM systems.	No live CRM/contact-form integration exists in the candidate prototype.
P2 - Scale	Add queueing, job status, background processing and load testing if inquiry volume requires it.	Multi-agent generation can take longer than a normal web request.

Proposed pilot and acceptance measures

Start with an architect-led discovery session and a small approved, de-identified sample. Run in shadow mode: produce drafts without sending them to prospects. Capture baseline and assisted active architect time, elapsed time to approved overview, requirement coverage, source-support rate, material edit rate and cost per accepted draft. Cynbit should set target thresholds after baseline measurement.

Go / no-go principle

Proceed beyond pilot only if the responsible architects judge drafts useful, critical claims are traceable, the human approval gate cannot be bypassed, data access is approved, and the measured process improves against an agreed baseline.

References

[1] Cynbit Contact / Project Intelligence: cynbit.in/contact | [2] Services and technology capabilities: cynbit.in/services
[3] Product ecosystem: cynbit.in/products | [4] AI agents and HITL article (13 Oct 2025): cynbit.in/blog/ai-agents-business-operations
[5] AI/ML Engineer Intern role: cynbit.in/careers/ai-ml-engineer-intern | [6] Careers / operating model: cynbit.in/careers
[7] Implementation evidence: linked repository README.md, docs/architecture.md, docs/evaluation.md, orchestration/workflow.py, rag/retrieval.py, services/llm_client.py and tests/. Public web sources accessed 31 October 2026.